TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN



NGUYỄN GIA PHÁT – 524H0117
HUỲNH KỲ PHONG – 524H0119

BÁO CÁO GIỮA KỲ
PHÁT TRIỂN ỨNG DỤNG DI ĐỘNG
ĐỀ TÀI 01
XÂY DỰNG ỨNG DỤNG ANDROID HỖ TRỢ HOẠT ĐỘNG NGOẠI TUYẾN VÀ ĐỒNG BỘ DỮ LIỆU TỰ ĐỘNG

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN



NGUYỄN GIA PHÁT – 524H0117
HUỲNH KỲ PHONG – 524H0119

BÁO CÁO GIỮA KỲ
PHÁT TRIỂN ỨNG DỤNG DI ĐỘNG
ĐỀ TÀI 01
XÂY DỰNG ỨNG DỤNG ANDROID HỖ TRỢ HOẠT ĐỘNG NGOẠI TUYẾN VÀ ĐỒNG BỘ DỮ LIỆU TỰ ĐỘNG

Người hướng dẫn
Ts. Lê Văn Vang

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

LỜI CẢM ƠN

Chúng em xin chân thành cảm ơn đến *Ban giám hiệu trường Đại học Tôn Đức Thắng* vì đã tạo điều kiện tối ưu về cơ sở vật chất và hệ thống thư viện hiện đại, đa dạng các loại thiết bị, sách báo giúp chúng em có thể tìm kiếm, nghiên cứu thông tin một cách thuận tiện nhất để hoàn thành bài báo cáo này.

Chúng em xin chân thành cảm ơn Ts. Lê Văn Vang, giảng viên bộ môn, đã rất nhiệt tình, hỗ trợ, và giúp đỡ chúng em trong việc học. Thầy đã chỉ cho chúng em biết nhiều cách thức làm bài hay, đồng thời đưa ra các gợi ý để chúng em có nhiều cách tư duy về một bài toán. Thầy đã luôn hỗ trợ hết mình khi chúng em cần và luôn giải thích kĩ càng và dễ hiểu. Một lần nữa, chúng em chân thành cảm ơn và biết ơn thầy.

TP. Hồ Chí Minh, ngày 4 tháng 11 năm 2026

Tác giả

PHONG

Huỳnh Kỳ Phong

PHAT

Nguyễn Gia Phát

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH
TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Chúng em xin cam đoan đây là công trình nghiên cứu của riêng chúng em và được sự hướng dẫn khoa học của Ts. Lê Văn Vang. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong Dự án còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào chúng em xin hoàn toàn chịu trách nhiệm về nội dung Dự án của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do chúng em gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, ngày 4 tháng 11 năm 2026

Tác giả

PHONG

Huỳnh Kỳ Phong

PHAT

Nguyễn Gia Phát

BÁO CÁO GIỮA KỲ

TÓM TẮT

Đề tài xây dựng ứng dụng quản lý công việc cá nhân trên Android theo kiến trúc Offline-First, tập trung vào khả năng thao tác khi mất Internet, lưu trữ cục bộ, tự động đồng bộ hai chiều với máy chủ và xử lý xung đột phiên bản. Ứng dụng được phát triển bằng Kotlin, Room, mô hình MVVM kết hợp Repository, Retrofit, Kotlin Coroutines/Flow và WorkManager; phía máy chủ sử dụng FastAPI cùng SQLite.

Khi người dùng thêm, sửa, đánh dấu hoàn thành hoặc xóa công việc, ứng dụng ghi thay đổi vào Room trước và cập nhật giao diện ngay. Các thay đổi chưa đồng bộ được theo dõi bằng trường syncStatus và chuyển lên server khi có điều kiện phù hợp. Hệ thống sử dụng UUID ổn định, serverVersion, Idempotency-Key và cơ chế xóa logic (tombstone) để hạn chế sai lệch dữ liệu và thao tác bị lặp. Nếu cập nhật có phiên bản cũ, máy chủ trả về HTTP 409 và ứng dụng xử lý theo điều kiện về thời gian cập nhật và trạng thái xóa.

Để đánh giá chương trình, nhóm tổ chức các kịch bản TC01–TC09 và ghi lại 32 ảnh minh chứng. Bộ Android Instrumented Tests ghi nhận 23/23 bài kiểm thử đạt; bộ kiểm thử FastAPI ghi nhận 26/26 đạt. Trong một lượt đo TC09, 20 công việc được đồng bộ trong 2,165 giây. Các kết quả này cho thấy hệ thống đáp ứng những tình huống được thử nghiệm, song chưa đủ để khẳng định hiệu năng trên mọi thiết bị, điều kiện mạng hay tình huống đồng thời thực tế.

Từ khóa: Offline-First, Kotlin, Android, Room, MVVM, Retrofit, WorkManager, FastAPI, SQLite, đồng bộ hai chiều, conflict resolution, idempotency.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU 1

1.1 Bối cảnh và vấn đề 1

1.2 Mục tiêu 1

1.3 Phạm vi triển khai 1

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 3

2.1 Kiến trúc Offline-First 3

2.2 MVVM và Repository Pattern 3

2.3 Room, Entity, DAO và SQLite 3

2.4 REST API, Retrofit và các mã HTTP 4

2.5 Coroutines, suspend, Flow và StateFlow 4

2.6 WorkManager và tác vụ nền 4

2.7 Đồng bộ hai chiều: Push và Pull 5

2.8 Xung đột và chiến lược giải quyết 5

2.9 Tính nhất quán và Idempotency 5

2.10 Quan điểm kiểm thử 6

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 7

3.1 Yêu cầu chức năng 7

3.2 Yêu cầu phi chức năng 7

3.3 Sơ đồ Use Case 8

3.4 Kiến trúc tổng thể 9

3.5 Sơ đồ thành phần 9

3.6 Thiết kế cơ sở dữ liệu 11
3.7 Thiết kế luồng đồng bộ 12
CHƯƠNG 4. HIỆN THỰC ANDROID VÀ FASTAPI 13
4.1 Cấu trúc mã nguồn Android 13
4.2 Quy trình cập nhật công việc từ giao diện 14
4.3 Thiết kế API máy chủ 14
4.4 Môi trường chạy, kết nối và timeout 15
4.5 Sản phẩm source và quản lý phiên bản 15
CHƯƠNG 5. MÔ-ĐUN ĐỒNG BỘ VÀ GIẢI QUYẾT XUNG ĐỘT 16
5.1 Vòng đời trạng thái một công việc 16
5.2 Thuật toán Push có điều kiện 16
5.3 Thuật toán Pull và hợp nhất 17
5.4 Xử lý tombstone và yêu cầu bị mất phản hồi 17
5.5 Phát hiện và xử lý xung đột 17
5.6 Retry, unique work và ngăn trùng lặp 18
CHƯƠNG 6. BỘ KIỂM THỬ VÀ KẾT QUẢ 19
6.1 Cách tổ chức kiểm thử 19
6.2 Bảng đối chiếu TC01–TC09 19
6.3 Kết quả tự động 20
6.4 Minh chứng thao tác online/offline 21
6.5 Minh chứng retry, lặp và xung đột 22
6.6 Minh chứng xóa offline và đồng bộ nhiều bản ghi 23
CHƯƠNG 7. THỰC NGHIỆM, ĐÁNH GIÁ VÀ HẠN CHẾ 24

7.1 Môi trường và dữ liệu thử nghiệm 24
7.2 Kết quả đồng bộ TC09 trên Android 24
7.3 Benchmark API FastAPI riêng biệt 25
7.4 Đánh giá tính nhất quán 25
7.5 Hạn chế.....26
7.6 Hướng phát triển 26
CHƯƠNG 8. SẢN PHẨM BÀN GIAO VÀ KẾT LUẬN 27
8.1 Danh sách sản phẩm.....27
8.2 Hướng dẫn tái hiện các kịch bản cốt lõi.....27
8.3 Địa chỉ GitHub và video demo.....28
8.4 Kết luận 28
TÀI LIỆU THAM KHẢO 30

CHƯƠNG 1. GIỚI THIỆU

1.1 Bối cảnh và vấn đề

Nhiều ứng dụng di động phụ thuộc trực tiếp vào mạng để tải và ghi dữ liệu. Khi thiết bị vào khu vực tín hiệu yếu, mất kết nối hoặc máy chủ tạm ngừng phản hồi, thao tác của người dùng có thể bị gián đoạn. Nếu ứng dụng chỉ giữ trạng thái trên giao diện mà không lưu cục bộ, dữ liệu vừa nhập dễ bị mất khi ứng dụng bị đóng. Vì vậy, bài toán không chỉ là hiển thị danh sách công việc mà còn bảo đảm vòng đời của những thay đổi chưa được chuyển lên máy chủ.

Trong một ứng dụng quản lý công việc, người dùng cần tạo, chỉnh sửa và hoàn thành công việc tại bất kỳ thời điểm nào. Mô hình Offline-First giải quyết vấn đề bằng cách ưu tiên nguồn dữ liệu trên thiết bị, trong khi việc trao đổi với máy chủ được thực hiện bất đồng bộ. Cách tiếp cận này nâng cao khả năng sử dụng khi mất mạng nhưng đặt ra các bài toán về nhận diện thay đổi, xử lý yêu cầu bị gửi lại, xóa logic, xung đột dữ liệu và tính nhất quán.

1.2 Mục tiêu

- Hiển thị và quản lý công việc từ Room kể cả khi máy chủ không truy cập được.
- Hỗ trợ thêm, sửa, xóa, đánh dấu hoàn thành và cập nhật giao diện tức thì.
- Thực hiện đồng bộ hai chiều: đẩy thay đổi cục bộ và tải dữ liệu máy chủ.
- Lập lịch đồng bộ tự động bằng WorkManager, có retry và nút đồng bộ thủ công.
- Phát hiện xung đột phiên bản, xử lý xóa logic và yêu cầu đồng bộ được gửi lại.
- Kiểm thử bằng TC01–TC09, tổng hợp ảnh minh chứng và đo hiệu năng của một số tình huống.

1.3 Phạm vi triển khai

Đề tài được triển khai dưới dạng ứng dụng Android một người dùng để minh họa kỹ thuật đồng bộ, không phải hệ thống quản lý dự án doanh nghiệp. Máy chủ

FastAPI chạy trong môi trường thử nghiệm cục bộ, sử dụng SQLite; ứng dụng giao tiếp qua HTTP/JSON thông qua ADB reverse trên Android Emulator. Trong các thử nghiệm xung đột, Swagger UI đóng vai trò mô phỏng thiết bị/client thứ hai. Hiện hệ thống chưa triển khai xác thực người dùng hay hạ tầng đám mây công khai.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 Kiến trúc Offline-First

Offline-First là định hướng thiết kế trong đó ứng dụng có thể đáp ứng các thao tác cốt lõi ngay cả khi không có mạng. Trong dự án, Room giữ vai trò nguồn dữ liệu trực tiếp cho giao diện (local source of truth); việc ghi dữ liệu được xác nhận ở mức cục bộ trước, sau đó ứng dụng mới gửi dữ liệu đến API. Điểm khác biệt so với mô hình API-first là trải nghiệm thêm/sửa công việc không phải chờ máy chủ trả lời thành công [1].

Ưu điểm là độ phản hồi của giao diện tốt, dữ liệu vẫn truy cập được khi mất mạng và người dùng có thể tiếp tục làm việc. Đổi lại, hệ thống phải lưu trạng thái chờ đồng bộ, xử lý lỗi mạng, chấp nhận trạng thái dữ liệu tạm thời khác biệt giữa thiết bị và server và có quy tắc rõ ràng khi hai bên cùng sửa.

2.2 MVVM và Repository Pattern

MVVM tách thành ba vai trò: View trình bày giao diện và tiếp nhận thao tác; ViewModel tổ chức trạng thái của màn hình; Model/Repository truy cập và phối hợp dữ liệu. Trong source, MainActivity cùng TaskAdapter phụ trách màn hình; TaskViewModel cung cấp StateFlow danh sách công việc và SyncUiState; TaskRepository xử lý ghi cục bộ; SyncRepository phụ trách đồng bộ với máy chủ [2].

Repository Pattern giúp giới hạn trách nhiệm của giao diện: màn hình không trực tiếp tạo câu lệnh SQL hay tự gửi HTTP. Khi cần thay đổi cơ chế đồng bộ, nhóm có thể sửa tầng dữ liệu mà không phải thiết kế lại toàn bộ giao diện.

2.3 Room, Entity, DAO và SQLite

SQLite là hệ quản trị cơ sở dữ liệu quan hệ nhúng được sử dụng trong Android và server mẫu [8]. Trên Android, Room cung cấp lớp trừu tượng để khai báo Entity, DAO và các truy vấn có kiểm tra lúc biên dịch [3]. TaskEntity ánh xạ vào bảng tasks. TaskDao có truy vấn Flow để theo dõi những bản ghi chưa xóa, các thao tác insert/update và những truy vấn cập nhật trạng thái đồng bộ có điều kiện.

Việc lưu trữ bằng Room giúp dữ liệu tồn tại sau khi đóng và mở lại ứng dụng. Trường updatedAt ghi nhận thời điểm thay đổi; trường serverVersion theo dõi phiên bản server gần nhất; trường isDeleted tạo dấu vết xóa logic; trường syncStatus phân biệt công việc mới thay đổi, đang gửi, đã gửi, lỗi hoặc xung đột.

2.4 REST API, Retrofit và các mã HTTP

REST API trao đổi đối tượng công việc qua HTTP/JSON. Retrofit chuyển khai báo phương thức Kotlin thành các request HTTP, kết hợp OkHttp để thiết lập timeout và xử lý mạng [6]. Các thao tác chính tương ứng với GET, POST, PUT và DELETE. Trong dự án, GET /tasks truy vấn công việc; POST /tasks tạo mới; PUT /tasks/{task_id} cập nhật; DELETE /tasks/{task_id} xóa logic. Ở phía máy chủ, nhóm sử dụng FastAPI để xây dựng các API phục vụ việc tiếp nhận và xử lý yêu cầu đồng bộ dữ liệu [7].

HTTP 409 thể hiện xung đột, ví dụ expectedVersion không trùng serverVersion hiện có hoặc Idempotency-Key đã dùng cho một yêu cầu khác. HTTP 404 báo không tìm thấy bản ghi. Các lỗi kết nối IOException và lỗi HTTP được xử lý khác nhau để ứng dụng có thể bảo toàn dữ liệu chờ gửi và lập lịch retry phù hợp.

2.5 Coroutines, suspend, Flow và StateFlow

Kotlin Coroutines hỗ trợ lập trình bất đồng bộ, cho phép thực hiện truy vấn Room và request Retrofit mà không chặn giao diện. Những hàm DAO và API có thể khai báo suspend; trong khi đó Flow phát luồng danh sách công việc khi database thay đổi. TaskViewModel sử dụng stateIn để chuyển Flow thành StateFlow phục vụ quan sát trên màn hình. MainActivity collect dữ liệu trong repeatOnLifecycle để gắn vòng đời của việc thu thập với Activity [5].

2.6 WorkManager và tác vụ nền

WorkManager lập lịch công việc nền có ràng buộc, thích hợp cho các tác vụ phải được thử lại khi điều kiện hệ thống cho phép [4]. Trong source, SyncScheduler yêu cầu NetworkType.CONNECTED, khai báo one-time work và periodic work, sử

dụng BackoffPolicy.EXPONENTIAL với độ trễ ban đầu 30 giây; tác vụ định kỳ đặt chu kỳ 15 phút. SyncWorker chạy coroutine đồng bộ, ghi log SyncMetrics và trả Result.retry() khi có lỗi thích hợp.

WorkManager không cam kết chạy ngay tức khắc khi có mạng; thời điểm thực thi phụ thuộc lịch của hệ điều hành. Trong bài kiểm thử, cần phân biệt việc có đường ADB reverse tới server với trạng thái kết nối mạng mà hệ điều hành nhận biết.

2.7 Đồng bộ hai chiều: Push và Pull

Đồng bộ hai chiều gồm hai phần. Push đọc những bản ghi chưa đồng bộ trong Room và gửi yêu cầu tạo, sửa hoặc xóa đến server. Pull lấy danh sách hiện có ở server và cập nhật bản ghi cục bộ theo điều kiện, bảo đảm không vô tình ghi đè thay đổi local chưa gửi [1]. Đồng bộ hoàn tất khi hai nguồn cùng phản ánh những thay đổi đã được chấp nhận theo chính sách xử lý xung đột.

2.8 Xung đột và chiến lược giải quyết

Xung đột xảy ra khi dữ liệu được sửa độc lập ở hai nơi trước khi đồng bộ. Dự án sử dụng Optimistic Concurrency Control ở API: khi PUT/DELETE, client cung cấp expectedVersion; server so sánh với serverVersion hiện tại và trả HTTP 409 khi lệch. Phía Android lấy phiên bản mới, so sánh trạng thái xóa và updatedAt. Server thắng khi đã xóa từ xa trong lúc local chưa xóa hoặc khi remote.updatedAt lớn hơn hay bằng local.updatedAt; trong trường hợp còn lại, Android có thể gửi lại thay đổi của mình dựa trên phiên bản mới.

Đây là chính sách theo thời điểm cập nhật có ưu tiên cho xóa từ server, không phải bảo đảm hòa giải mọi sửa đổi có chủ đích. Nếu thời gian thiết bị lệch đồng hồ hoặc cả hai phía cùng có thay đổi quan trọng, chính sách có thể loại bỏ một phiên bản dữ liệu. Phần đánh giá cần nêu rõ đánh đổi này.

2.9 Tính nhất quán và Idempotency

Trong mạng không ổn định, server có thể hoàn thành thao tác nhưng phản hồi chưa đến thiết bị. Nếu client gửi lại cùng yêu cầu mà server xử lý lại như một yêu cầu

mới, có thể sinh dữ liệu trùng hoặc cập nhật quá mức. Hệ thống dùng UUID ổn định cho từng công việc; header Idempotency-Key cho POST/PUT/DELETE; phía server lưu khóa cùng fingerprint và responseJson trong bảng request_log. Nếu yêu cầu có cùng khóa và nội dung, server phát lại kết quả đã lưu thay vì tạo thay đổi mới.

Nếu một khóa được sử dụng lại với nội dung khác, server báo HTTP 409. Tuy nhiên cơ chế Idempotency-Key chỉ phát huy tác dụng khi client duy trì khóa ổn định cho cùng một thao tác; việc tạo yêu cầu mới có khóa khác cần được kiểm soát trong thuật toán. Dữ liệu cục bộ chỉ được đánh dấu SYNCED sau khi ghi nhận kết quả server phù hợp.

2.10 Quan điểm kiểm thử

Kiểm thử Offline-First cần kiểm tra không chỉ chức năng CRUD mà cả trạng thái khi mất kết nối, sau khi khôi phục mạng, sau khi đóng ứng dụng, và khi server nhận yêu cầu lặp. Bên cạnh kiểm thử tự động của DAO, Repository và API, nhóm dùng các kịch bản TC01–TC09 theo đề gốc để thu thập ảnh, log và đo thời gian. Phép đo thời gian cần ghi rõ đối tượng đo là đồng bộ Android hay chỉ benchmark API server.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Yêu cầu chức năng

ID	Yêu cầu	Mô tả
FR01	Xem danh sách	Đọc công việc chưa xóa từ Room và hiển thị số lượng trên giao diện.
FR02	Thêm công việc	Thêm tiêu đề và mô tả; lưu vào Room kể cả khi offline.
FR03	Sửa công việc	Sửa tiêu đề/mô tả, cập nhật updatedAt, chuyển về PENDING.
FR04	Hoàn thành công việc	Đảo trạng thái isCompleted và lên lịch đồng bộ.
FR05	Xóa công việc	Đánh dấu isDeleted = true, ẩn khỏi danh sách nhưng giữ tombstone.
FR06	Đồng bộ thủ công	Người dùng nhấn Đồng bộ ngay để chạy push/pull.
FR07	Kiểm tra server	Gửi GET /health và hiển thị trạng thái kết nối.
FR08	Đồng bộ tự động	Lên lịch WorkManager có ràng buộc mạng và retry.
FR09	Xử lý xung đột	Phát hiện 409 và xử lý theo serverVersion, updatedAt, isDeleted.

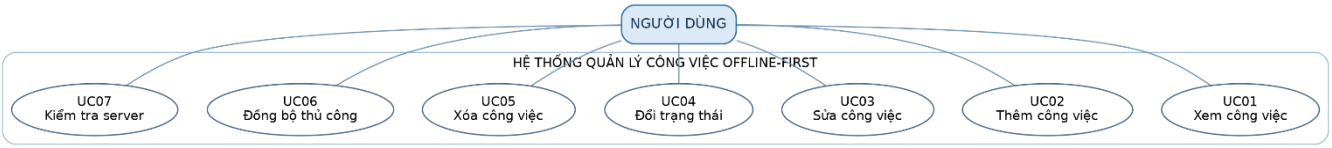
3.2 Yêu cầu phi chức năng

Nhóm	Tiêu chí thiết kế
Khả dụng khi offline	Thao tác CRUD và xem danh sách không phụ thuộc thời điểm server trả lời.
Độ bền dữ liệu	Room lưu dữ liệu qua các lần khởi động lại Activity/ứng dụng.

Đáp ứng giao diện	Sử dụng Coroutines và Flow để tránh chặn luồng giao diện.
Tính nhất quán	Phiên bản server và trạng thái pending được theo dõi để hạn chế mất cập nhật.
Khả năng phục hồi	WorkManager xử lý công việc nền và retry khi gặp sự cố tạm thời.
Tính lặp an toàn	UUID và Idempotency-Key giúp hạn chế tác dụng phụ của request bị gửi lại.
Giới hạn bảo mật	Backend demo dùng HTTP nội bộ và chưa có xác thực; không phù hợp công khai ra Internet.

3.3 Sơ đồ Use Case

Actor chính là Người dùng. Các Use Case trực tiếp gồm xem, thêm, sửa, đánh dấu hoàn thành, xóa công việc, đồng bộ thủ công và kiểm tra server. Đồng bộ nền là hành vi tự động của hệ thống nên trình bày trong sơ đồ luồng/kiến trúc thay vì xem WorkManager như một actor người dùng.

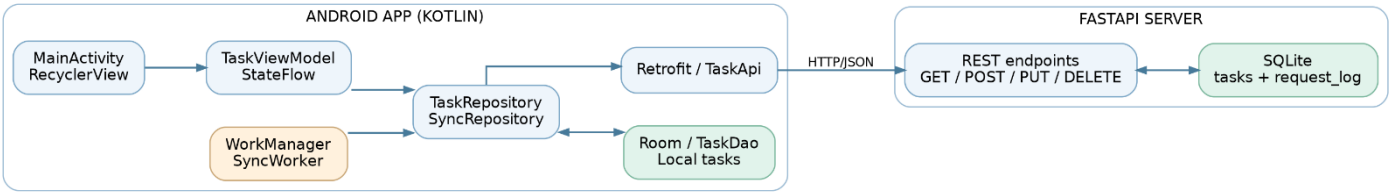


Hình 3.1. Sơ đồ Use Case mức chức năng của ứng dụng (dựng theo MainActivity/TaskViewModel).

Use Case	Tiền điều kiện	Luồng chính / kết quả
UC02 – Thêm công việc	Ứng dụng đang mở	Nhập tiêu đề, mô tả → kiểm tra tiêu đề → insert Room, PENDING → danh sách cập nhật → lên lịch sync.

UC03 – Sửa công việc	Có công việc chưa xóa	Chọn sửa → nhập nội dung mới → cập nhật updatedAt, PENDING → lưu Room → lên lịch sync.
UC05 – Xóa công việc	Có công việc đang hiển thị	Chọn xóa → xác nhận → isDeleted=1, PENDING → ẩn khỏi danh sách → đợi đẩy lên server.
UC06 – Đồng bộ thủ công	Người dùng mở app; server có thể sẵn sàng hoặc lỗi	Nhấn ĐỒNG BỘ NGAY → SyncRepository push/pull → hiển thị kết quả, thời gian hoặc lỗi.

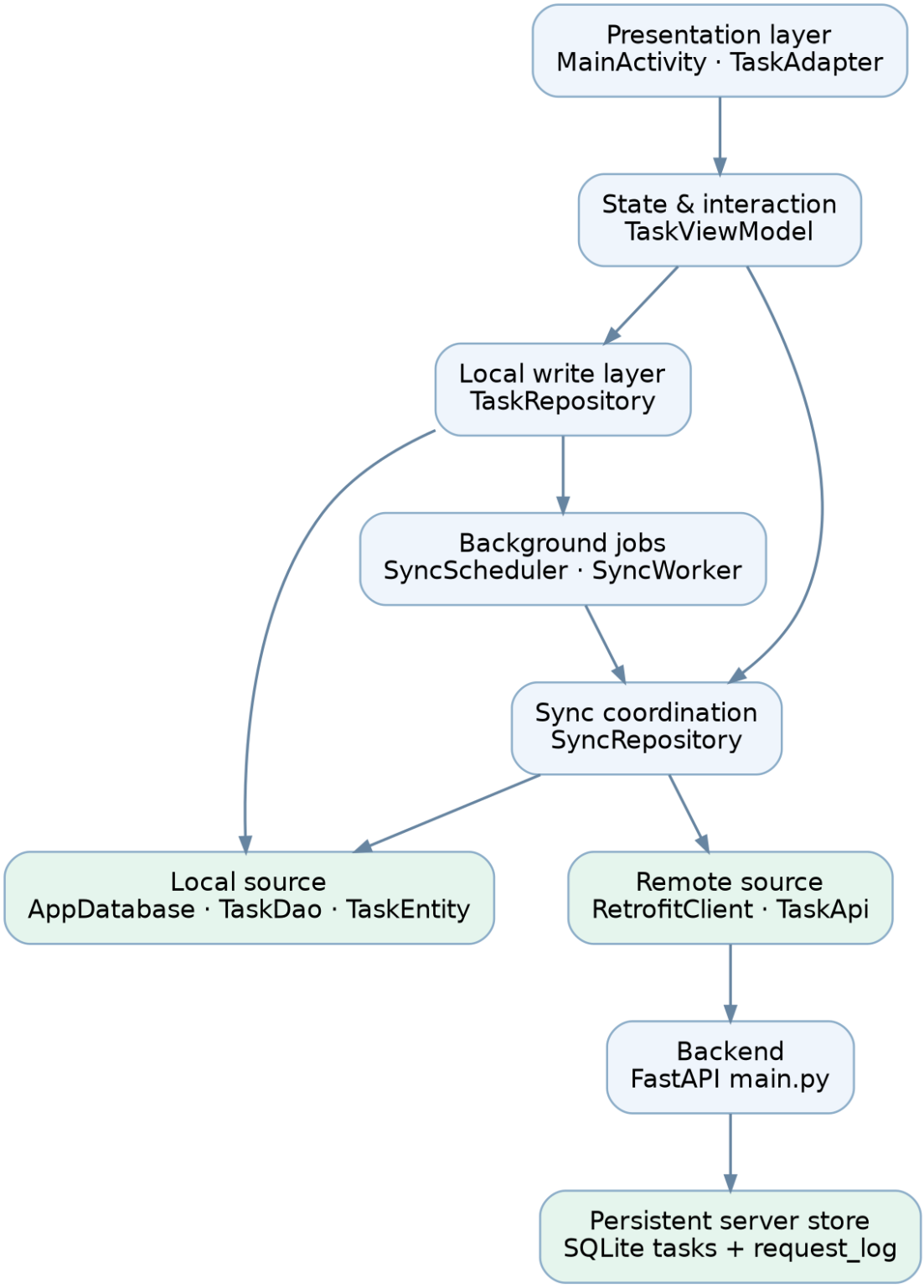
3.4 Kiến trúc tổng thể



Hình 3.2. Sơ đồ tổng quan Android – FastAPI – SQLite và các luồng dữ liệu.

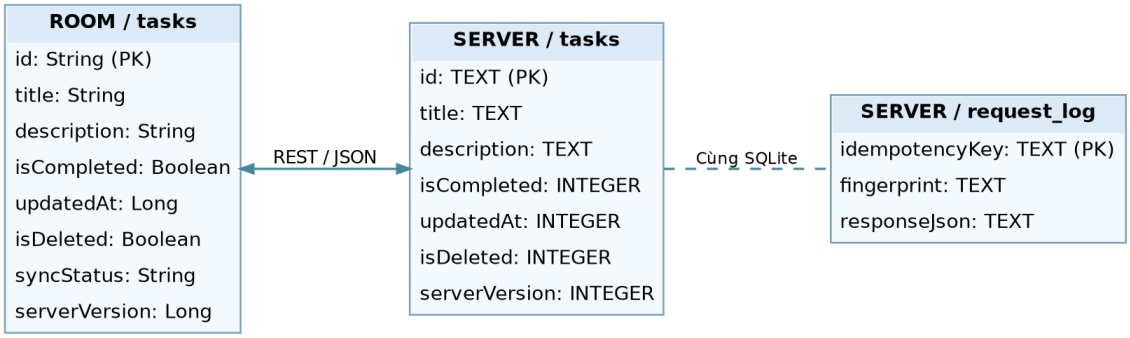
Trên Android, MainActivity hiển thị và thu thập tương tác; TaskViewModel chuyển thao tác đến Repository. TaskRepository ghi dữ liệu trước vào Room, còn SyncRepository điều phối các request đến Retrofit. WorkManager kích hoạt các lần đồng bộ tự động. Ở phía server, FastAPI cung cấp các endpoint và lưu dữ liệu trong SQLite. Đây là kiến trúc client–server với nguồn dữ liệu cục bộ cho giao diện.

3.5 Sơ đồ thành phần



Hình 3.3. Sơ đồ thành phần chính, phản ánh vị trí tương đối của các lớp trong source.

3.6 Thiết kế cơ sở dữ liệu

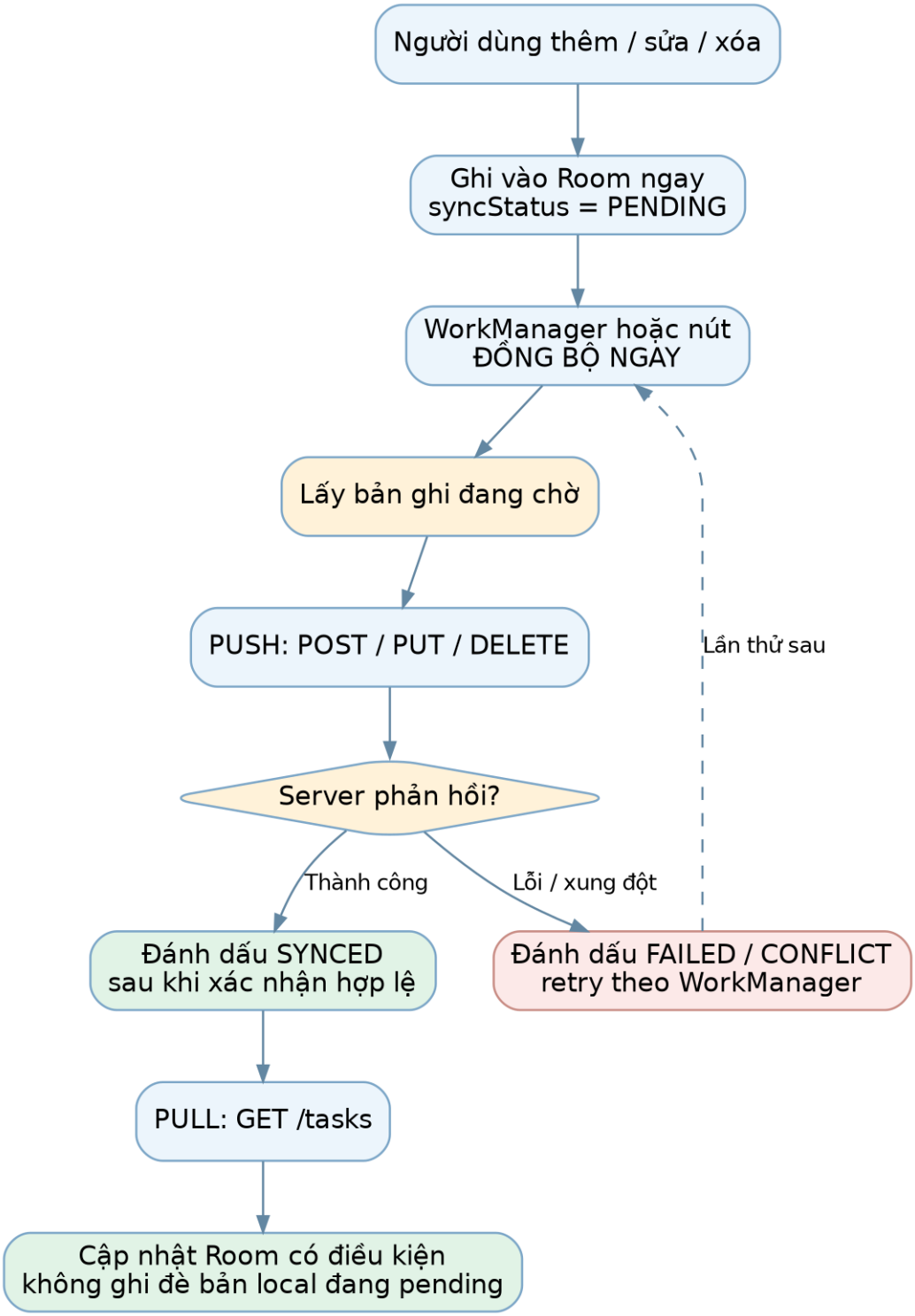


Hình 3.4. Thiết kế dữ liệu Room và SQLite server (không phải quan hệ khóa ngoại giữa các database).

Trường TaskEntity	Kiểu	Vai trò
id	String / UUID	Khóa chính ổn định, sử dụng xuyên suốt client–server.
title	String	Tên công việc, không cho để trống khi nhập.
description	String	Nội dung mô tả công việc.
isCompleted	Boolean	Trạng thái hoàn thành hay chưa.
updatedAt	Long	Mốc thời gian cập nhật, đơn vị mili giây.
isDeleted	Boolean	Xóa logic, dùng tombstone.
syncStatus	String	PENDING, SYNCING, SYNCED, FAILED, CONFLICT.
serverVersion	Long	Phiên bản server đã biết của bản ghi.

Database của ứng dụng dùng Room tên tasks_database, version = 1 và một bảng TaskEntity. SQLite server có bảng tasks gồm cùng các trường nghiệp vụ nhưng không có syncStatus, và bảng request_log lưu idempotencyKey, fingerprint, responseJson. Bảng request_log phục vụ phát lại phản hồi của request gửi lặp; không phải bảng thông tin người dùng.

3.7 Thiết kế luồng đồng bộ



Hình 3.5. Lưu đồ đồng bộ mức tổng quát: ghi local → push → pull → cập nhật Room.

Các thao tác thêm/sửa/xóa không đợi mạng. Khi có yêu cầu đồng bộ, hệ thống tìm bản ghi PENDING/FAILED/SYNCING/CONFLICT, đánh dấu SYNCING trước khi gửi, phân loại POST/PUT/DELETE theo trạng thái dữ liệu, sau đó xác nhận serverVersion nếu thao tác được server chấp nhận. Tiếp theo hệ thống gọi GET /tasks để tải dữ liệu và hợp nhất một cách có điều kiện. Khi lỗi tạm thời, trạng thái chờ được giữ lại cho lần retry.

CHƯƠNG 4. HIỆN THỰC ANDROID VÀ FASTAPI

4.1 Cấu trúc mã nguồn Android

File / nhóm file	Vai trò thực tế
MainActivity.kt	Tạo màn hình, thêm/sửa/xóa, hoàn thành, kiểm tra server và đồng bộ ngay.
TaskAdapter.kt	Hiển thị danh sách công việc và các sự kiện tương tác từng dòng.
TaskViewModel.kt	Cung cấp StateFlow danh sách, trạng thái đồng bộ, gọi Repository.
TaskEntity.kt, TaskDao.kt, AppDatabase.kt	Cấu trúc và truy vấn Room, cập nhật trạng thái cục bộ.
TaskRepository.kt	Thực thi CRUD local và gọi SyncScheduler.
SyncRepository.kt	Đẩy/kéo dữ liệu, xử lý 404/409, tombstone, đánh dấu SYNCED có điều kiện.
RetrofitClient.kt, TaskApi.kt	Định nghĩa API, URL nội bộ, timeout và các kiểu request/response.
SyncScheduler.kt, SyncWorker.kt	WorkManager one-time/periodic, constraints, retry và log thời gian.

4.2 Quy trình cập nhật công việc từ giao diện

Khi người dùng nhấn Thêm, MainActivity kiểm tra tiêu đề không trống rồi gọi TaskViewModel.addTask(). ViewModel chạy coroutine và gọi TaskRepository.addTask(). Repository tạo TaskEntity có id UUID mới, syncStatus=PENDING, ghi vào Room và yêu cầu SyncScheduler.enqueue(). Danh sách Flow tự phát thay đổi đến ViewModel và màn hình, không cần chờ server.

Sửa, đánh dấu hoàn thành và xóa tuân theo nguyên tắc tương tự. Xóa không loại bỏ vật lý bản ghi; TaskDao cập nhật isDeleted=1, khiến bản ghi không còn được trả về bởi getAllTasks(), song vẫn tồn tại trong database để lần đồng bộ tiếp theo truyền sự kiện xóa đến máy chủ.

4.3 Thiết kế API máy chủ

Phương thức	Endpoint	Kết quả chính
GET	/health	Kiểm tra API: status = ok.
GET	/tasks	Lấy danh sách, kể cả bản ghi server đã xóa logic.
GET	/tasks/{task_id}	Lấy một bản ghi theo UUID.
POST	/tasks	Tạo bản ghi có ID client cung cấp; HTTP 201 hoặc xung đột.
PUT	/tasks/{task_id}	Cập nhật nếu expectedVersion bằng serverVersion.
DELETE	/tasks/{task_id}?expectedVersion=...	Xóa logic nếu đúng phiên bản; giữ ID/tombstone.

FastAPI khai báo các mô hình TaskCreate, TaskUpdate và TaskOut. TaskCreate giới hạn title dài 1–200 ký tự và description tối đa 4000 ký tự; TaskUpdate cần expectedVersion từ 1 trở lên. Cơ chế tạo, cập nhật và xóa lưu

serverVersion và updatedAt. Server dùng transaction BEGIN IMMEDIATE khi xử lý request ghi để kiểm soát tính nguyên tử đối với thao tác ghi dữ liệu và lưu request_log.

API hỗ trợ Swagger UI tại http://127.0.0.1:8000/docs. Đây cũng là giao diện được sử dụng để minh họa dữ liệu server và mô phỏng client thứ hai khi kiểm thử xung đột.

4.4 Môi trường chạy, kết nối và timeout

Backend chạy localhost cổng 8000. Trong Android_Source, RetrofitClient đặt BASE_URL = http://127.0.0.1:8000/; do vậy khi chạy trên emulator phải cấu hình ADB reverse tcp:8000 tcp:8000 để chuyển lưu lượng về FastAPI trên máy tính. OkHttp đặt connectTimeout 10 giây, readTimeout 15 giây, writeTimeout 15 giây và callTimeout 25 giây. Đây là cấu hình trong source, không đồng nghĩa đã có số liệu kiểm thử riêng cho mọi loại timeout.

Hướng dẫn mở terminal tại FastAPI_Server

Sau khi khởi động server và emulator, mở http://127.0.0.1:8000/health để kiểm tra trạng thái ok. Android Studio mở dự án Android_Source và chạy app-debug.apk hoặc build từ source. Tài liệu README_CHAY_VA_TEST_CODE.md ở thư mục gốc là nguồn hướng dẫn chạy và kiểm thử chi tiết.

4.5 Sản phẩm source và quản lý phiên bản

Repository GitHub: https://github.com/nguyengiaphat695-coder/524H0117_524H0119_GiuaKi_DeTai1.git. Nguồn bàn giao gồm Android_Source, FastAPI_Server, APK/app-debug.apk, 32 ảnh trong MinhChung_KiemThu, báo cáo kết quả Android, file kiểm thử backend và benchmark. Trước khi nộp, nhóm kiểm tra quyền truy cập GitHub: nếu repository để Private cần mời giảng viên, hoặc chọn phương án chia sẻ được giảng viên chấp nhận.

CHƯƠNG 5. MÔ-ĐUN ĐỒNG BỘ VÀ GIẢI QUYẾT XUNG ĐỘT

5.1 Vòng đời trạng thái một công việc

Trạng thái	Ý nghĩa	Chuyển tiếp
PENDING	Có thay đổi cục bộ chưa được server xác nhận.	Được WorkManager hoặc người dùng chọn gửi.
SYNCING	Bản ghi đang được xử lý trong một lượt push.	Nếu nhận phản hồi hợp lệ: SYNCED; khi lỗi: FAILED/CONFLICT.
SYNCED	Bản ghi phản ánh phiên bản server được xác nhận.	Khi người dùng sửa mới: PENDING.
FAILED	Thao tác gửi gặp lỗi và có thể thử lại.	Lượt đồng bộ kế tiếp tìm lại bản ghi.
CONFLICT	Xung đột chưa thể giải quyết sau các lần thử giới hạn.	Được đưa vào danh sách chờ đồng bộ để xử lý lại.

5.2 Thuật toán Push có điều kiện

SyncRepository đọc danh sách getPendingForSync() theo updatedAt tăng dần và dùng markSyncing() có điều kiện để tránh ghi đè trạng thái của bản ghi đã bị người dùng sửa ở thời điểm khác. Dựa trên isDeleted và serverVersion, hệ thống quyết định gửi DELETE, POST hoặc PUT. Tên khóa Idempotency-Key được tạo từ task ID, thời điểm cập nhật và loại thao tác. Sau khi server trả về, confirmSynced() chỉ chuyển thành SYNCED nếu updatedAt vẫn khớp với phiên bản đang gửi; nếu người dùng đã sửa tiếp, trạng thái tiếp tục PENDING.

```
for task in pendingTasks:
    markSyncing(task.id, task.updatedAt)
    if task.isDeleted: request = DELETE
    elif task.serverVersion == 0: request = POST
```

```
else: request =
PUT(expectedVersion=task.serverVersion)
send(request,
idempotencyKey=stable_key_for_operation)
confirmSynced_only_if_sent_updatedAt_matches()
```

Đoạn mã giả giải thích thuật toán; không phải mã Kotlin sao chép nguyên văn.

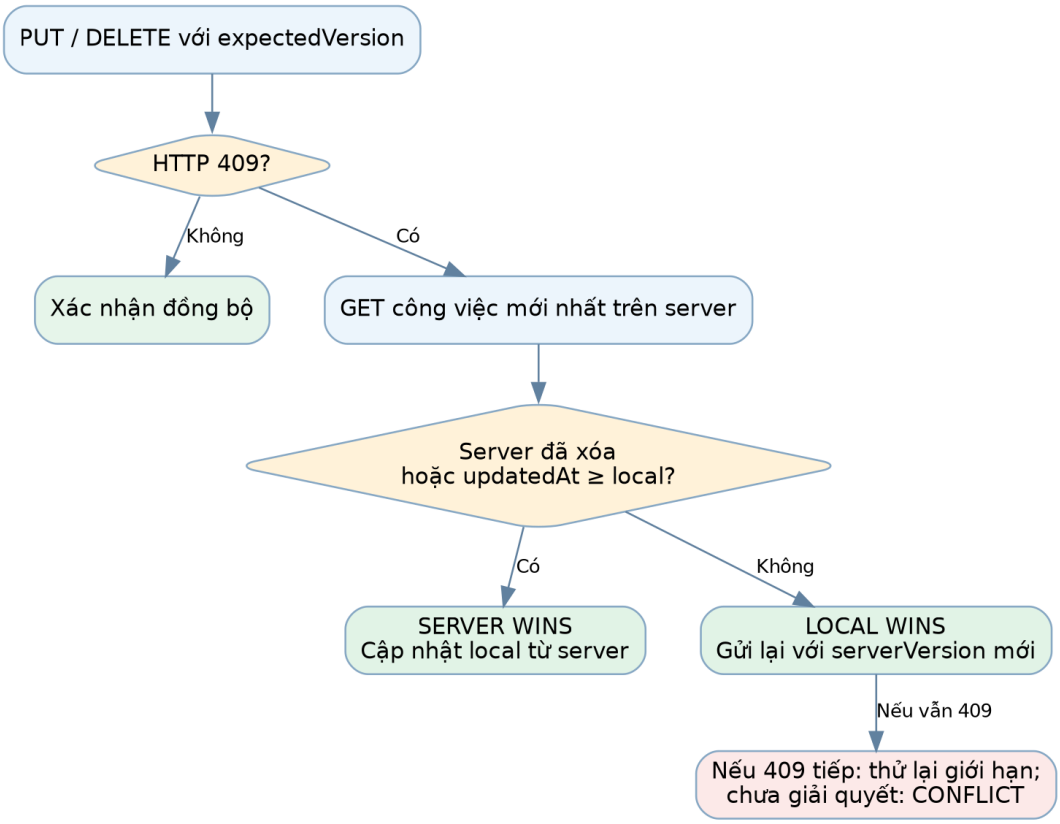
5.3 Thuật toán Pull và hợp nhất

Sau push, ứng dụng GET /tasks và duyệt từng công việc server. Nếu local chưa có bản ghi tương ứng, Room tạo bản ghi mới từ server. Nếu bản local đang SYNCED và remote.serverVersion mới hơn, Room cập nhật từ server. Trường hợp local đang PENDING/FAILED/CONFLICT không được thay thế tùy tiện bởi kết quả pull, giúp bảo vệ dữ liệu chưa gửi. Server trả về cả bản ghi đã bị xóa logic; thông tin isDeleted được giữ để ngăn đồng bộ nhầm dữ liệu đã xóa.

5.4 Xử lý tombstone và yêu cầu bị mất phản hồi

Đối với xóa local, TaskDao không xóa hàng khỏi bảng mà cập nhật isDeleted=1, syncStatus=PENDING. Khi push, DELETE truyền phiên bản server mà Android biết. Đối với trường hợp bản ghi vừa tạo rồi bị xóa khi request tạo đã được xử lý nhưng client chưa nhận được phản hồi, thuật toán còn kiểm tra lại trạng thái trên server bằng GET theo ID. Nếu remote có bản ghi chưa xóa, app gửi DELETE theo version mới; nếu remote đã xóa hoặc không tìm thấy, bản ghi local được xác nhận theo nhánh thích hợp. Cách xử lý này phục vụ tình huống mất phản hồi và sự kiện xóa, nhưng cần đối chiếu với bằng chứng kiểm thử hiện có trước khi viết thành nhận định tuyệt đối.

5.5 Phát hiện và xử lý xung đột



Hình 5.1. Luồng xử lý HTTP 409 và quyết định server thắng/Android thắng.

Khi server trả HTTP 409 cho PUT/DELETE do sai phiên bản, app GET /tasks/{id} để lấy phiên bản mới. Nếu remote.isDeleted và local chưa xóa, hoặc remote.updatedAt lớn hơn/bằng local.updatedAt, nhánh SERVER_WINS áp dụng dữ liệu server vào Room khi bản ghi local vẫn đúng trạng thái đang xử lý. Ngược lại, nhánh LOCAL_WINS có thể gửi lại dữ liệu local với expectedVersion mới. Mã nguồn giới hạn tối đa ba lượt xử lý xung đột; khi không thể hòa giải sẽ đánh dấu CONFLICT để tiếp tục xử lý về sau.

Nhóm thực nghiệm hai kịch bản: Server Wins khi dữ liệu server là phía được ưu tiên; Android Wins khi thay đổi Android có thời điểm cập nhật mới hơn. Client thứ hai trong thử nghiệm là Swagger UI, không phải hai điện thoại vật lý.

5.6 Retry, unique work và ngăn trùng lặp

SyncScheduler dùng enqueueUniqueWork và enqueueUniquePeriodicWork để quản lý lịch đồng bộ theo tên. One-time work sử dụng

ExistingWorkPolicy.APPEND_OR_REPLACE; periodic work dùng ExistingPeriodicWorkPolicy.KEEP. Khi SyncWorker nhận kết quả có errorMessage hoặc IOException, nó trả Result.retry(), còn WorkManager áp dụng exponential backoff. Không nên mô tả cơ chế này là bảo đảm đồng bộ xảy ra ngay lập tức sau khi mở mạng vì lịch chạy thực tế thuộc quyền hệ điều hành.

Phía server, request_log lưu khóa idempotency, dấu vân tay của nội dung và JSON phản hồi. Request hợp lệ bị gửi lặp với cùng khóa sẽ nhận kết quả cũ, giảm nguy cơ tạo bản ghi trùng. Cả khóa UUID ổn định và kiểm soát phiên bản đều tham gia quá trình bảo toàn dữ liệu.

CHƯƠNG 6. BỘ KIỂM THỬ VÀ KẾT QUẢ

6.1 Cách tổ chức kiểm thử

Nhóm sử dụng ba lớp minh chứng. Thứ nhất, bộ Android Instrumented Tests chạy trên emulator và kiểm tra DAO/Repository theo các ca đã lập trình. Thứ hai, bộ pytest kiểm tra API FastAPI và các tình huống dữ liệu trên SQLite tạm, không ghi đè tasks.db demo. Thứ ba, TC01–TC09 mô phỏng các tình huống sử dụng trên app, server Swagger, ADB reverse và log WorkManager; 32 ảnh chụp được đánh số liên tục từ 01 đến 32.

6.2 Bảng đối chiếu TC01–TC09

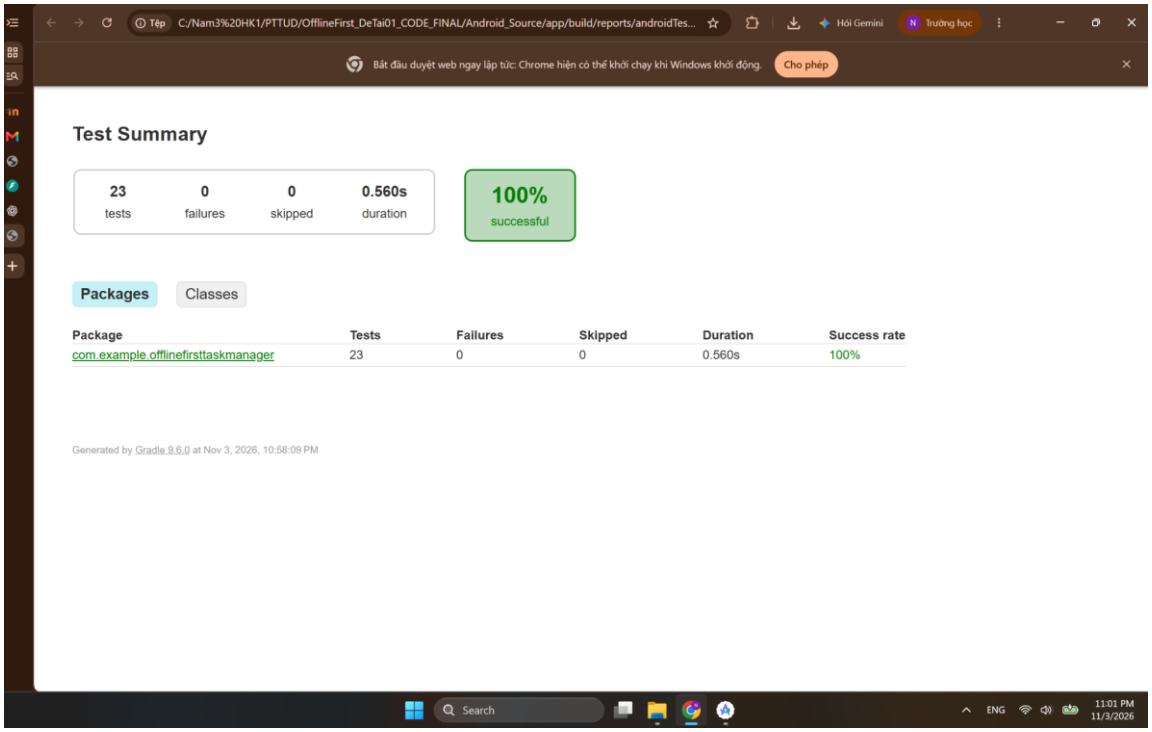
TC	Tình huống	Kỳ vọng / bằng chứng	Tài liệu ảnh
TC01	Tạo công việc online	App hiển thị và GET /tasks thấy cùng bản ghi.	02, 03
TC02	Tạo và sửa offline	Room lưu và hiển thị thay đổi khi server không tới được.	04, 05
TC03	Khôi phục kết nối	Thay đổi được đẩy lên server; ảnh và log cho thấy đồng bộ.	06, 07, 08

TC04	Đóng/mở app	Dữ liệu Room được giữ, có log đồng bộ nền.	09, 10, 11
TC05	Server không phản hồi	Chưa mất thay đổi local; trạng thái retry và khôi phục.	12–15
TC06	Gửi đồng bộ lặp	Không xuất hiện công việc trùng ngoài ý muốn.	16–18
TC07	Hai phía cùng sửa	Quan sát Server Wins và Android Wins trong thí nghiệm bằng Swagger.	19–25
TC08	Xóa offline	Tombstone được đẩy lên; công việc không tự xuất hiện lại.	26–29
TC09	Đồng bộ 20 bản ghi	Có log thời gian, 20/20 xử lý và không tạo trùng.	30–32

6.3 Kết quả tự động

Bộ kiểm thử	Kết quả đã ghi nhận	Nguồn
Android Instrumented Tests	23 tests; 0 failures; 0 skipped; 100%; 0,560 giây.	KetQua_KiemThu_Android/debug/index.html; ảnh 01
FastAPI pytest	26 passed trong 1,45 giây.	KET_QUA_BACKEND.txt; tests/test_api.py

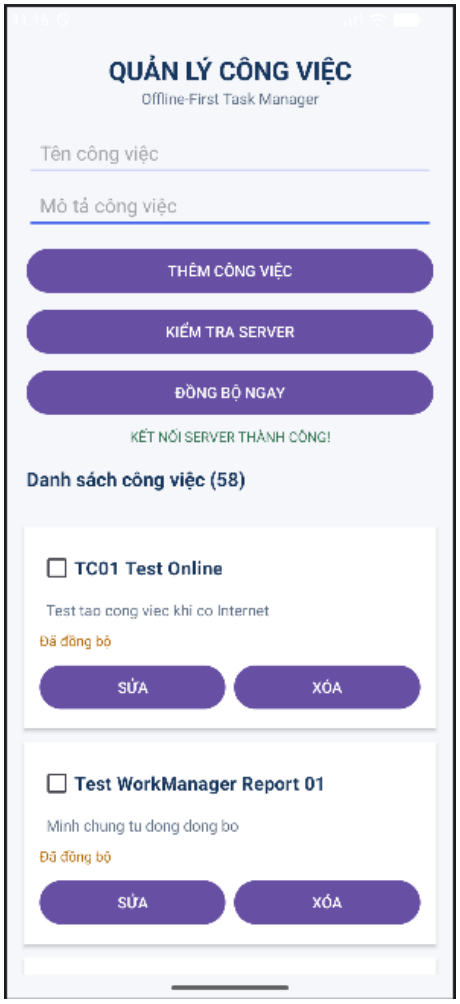
Kết quả pass của hai bộ test không có nghĩa là mọi kịch bản hệ thống phân tán, thời gian chờ hoặc tương tác từ hai thiết bị thực đều đã được thử đầy đủ. Bảng TC01–TC09 sử dụng ảnh và log làm minh chứng cho những đường đi đã thực hành trong môi trường nhóm.



Hình 6.1. Minh chứng Android Instrumented Tests: 23/23 bài kiểm thử đạt.

6.4 Minh chứng thao tác online/offline

TC01 cho thấy công việc được tạo trên Android và hiển thị trên server. Với TC02, đường chuyển tiếp ADB reverse tới FastAPI được ngắt để mô phỏng việc ứng dụng không thể kết nối đến máy chủ. Đây là mô phỏng mất đường kết nối API, không phải bằng chứng điện thoại bật chế độ máy bay. Khi khôi phục kết nối ở TC03, nhóm kiểm tra trạng thái Android, danh sách server và log của WorkManager.



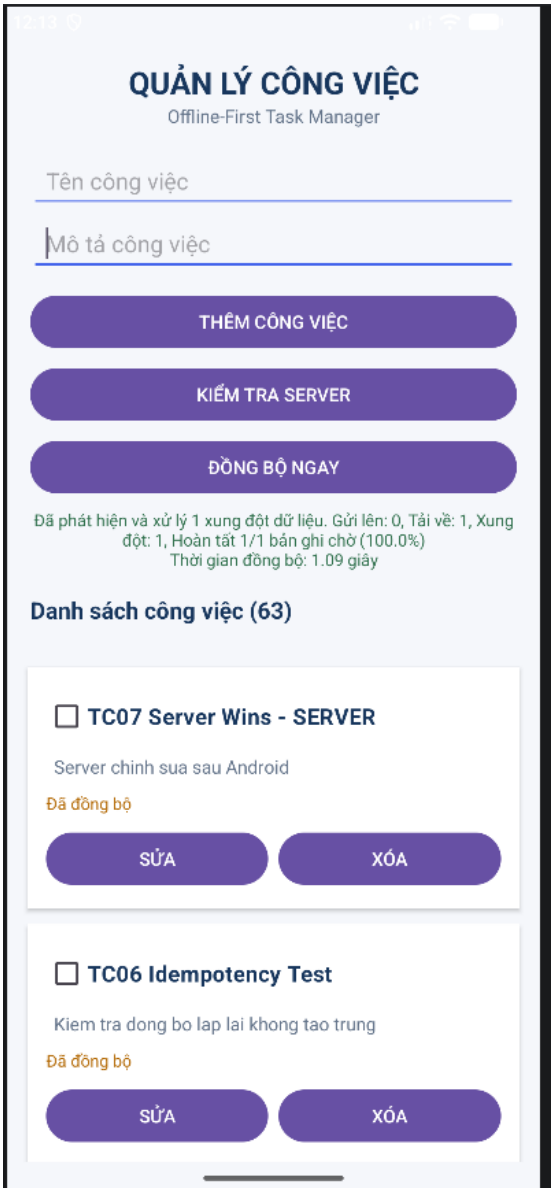
TC01 – Android online



TC02 – Tạo offline

6.5 Minh chứng retry, lặp và xung đột

TC05 kiểm tra khi server tạm không đáp ứng: thay đổi local vẫn còn; sau khi server phục hồi, hệ thống có đường retry và đồng bộ tiếp. TC06 sử dụng nhiều lần đồng bộ để kiểm tra không tạo thêm bản ghi trùng. TC07 gồm hai biến thể: Server Wins và Android Wins. Các ảnh 19–25 cho thấy quá trình chỉnh sửa Android/Swagger và kết quả quan sát được sau đồng bộ. Cần nêu rõ Swagger UI là một client mô phỏng thứ hai.



Hình 6.2. Minh chứng một tình huống Server Wins được chụp lại trong TC07.

6.6 Minh chứng xóa offline và đồng bộ nhiều bản ghi

TC08 được thực hiện bằng việc xóa công việc khi không thể gửi API, kiểm tra dữ liệu ẩn trong danh sách Android, trạng thái tombstone trên server sau kết nối lại và xác nhận công việc không xuất hiện lại. TC09 tạo 20 công việc và kiểm tra số lượng đồng bộ, thời gian và tính không trùng lặp. Toàn bộ 32 ảnh minh chứng cho các trường hợp kiểm thử TC01–TC09 được lưu trong thư mục MinhChung_KiemThu/ tại GitHub của nhóm, giúp người đọc đối chiếu kết quả thực nghiệm với từng kịch bản kiểm thử.



Hình 6.3. Log thực nghiệm đồng bộ nhiều công việc – nguồn ảnh TC09 của nhóm.

CHƯƠNG 7. THỰC NGHIỆM, ĐÁNH GIÁ VÀ HẠN CHẾ

7.1 Môi trường và dữ liệu thử nghiệm

Android được thử trên máy ảo Medium Phone API 37.2. FastAPI chạy cục bộ trên Windows, sử dụng Python, SQLite và Uvicorn, trong khi Android gọi API qua ADB reverse. Các bộ API test chạy với database SQLite tạm riêng. Dữ liệu thử nghiệm gồm các công việc demo online/offline, công việc sửa/xóa và tập 20 công việc cho TC09. Các phép đo chỉ phản ánh môi trường và lượt chạy đã ghi nhận, chưa phải benchmark đa thiết bị hoặc đa mạng.

7.2 Kết quả đồng bộ TC09 trên Android

Chỉ số	Kết quả đã ghi nhận	Cách hiểu
Số công việc trong lượt đo	20	Tập dữ liệu của TC09.
Số công việc đồng bộ	20/20	Theo ảnh/log thử nghiệm đã lưu.
Thời gian của lượt TC09	2,165 giây	Một lượt đo thực nghiệm Android.
Thông lượng suy ra	≈ 9,24 công việc/giây	20 / 2,165; chỉ là ước lượng từ một lượt đo.

Trùng lặp quan sát	Không ghi nhận trong TC09	Theo ảnh kiểm tra tên/ID sau đồng bộ.

Kết quả TC09 cho thấy ứng dụng xử lý 20 công việc trong một lượt thử nghiệm có thời gian khoảng 2,165 giây. Nếu chia số bản ghi cho thời gian, thông lượng xấp xỉ 9,24 công việc/giây. Đây không phải chỉ số bảo đảm cho thiết bị thật, tải cao hoặc mạng di động; cần nhiều lượt lặp và các điều kiện khác nhau mới có thể đánh giá hiệu năng có ý nghĩa thống kê.

7.3 Benchmark API FastAPI riêng biệt

Lượt	Request POST (kể cả gửi lại)	Bản ghi duy nhất	Lỗi	Thời gian (giây)	Kết quả
1	40	20	0	0,2057	PASS
2	40	20	0	0,1530	PASS
3	40	20	0	0,1843	PASS
4	40	20	0	0,1627	PASS
5	40	20	0	0,1737	PASS

Mỗi lượt benchmark API gửi 40 POST bao gồm các lần gửi lại tương ứng với 20 công việc, kết quả lưu 20 bản ghi duy nhất và không ghi nhận lỗi trong dữ liệu CSV. Thời gian của phép đo dao động từ 0,1530 đến 0,2057 giây trong 5 lượt. Đây là benchmark riêng của FastAPI/SQLite trên dữ liệu tạm, không được trình bày như thời gian WorkManager đồng bộ Android.

7.4 Đánh giá tính nhất quán

Qua các kịch bản thử nghiệm đã lưu, những hành vi trọng tâm gồm lưu được nội dung offline, đưa dữ liệu lên server khi kết nối khôi phục, không tạo bản ghi trùng khi đồng bộ lặp, xử lý được hai nhánh xung đột mô phỏng và xóa logic không làm công việc tự hiện lại. Những kết quả này hỗ trợ nhận định rằng các cơ chế thiết kế đã được thực thi đúng trong phạm vi thử nghiệm. Tuy vậy, chưa thể ngoại suy thành bảo

đảm tuyệt đối trên tất cả tình huống đồng thời và tất cả trạng thái hệ điều hành Android.

7.5 Hạn chế

- Chưa có minh chứng kiểm thử bằng hai thiết bị Android vật lý đồng thời; TC07 dùng Swagger UI để mô phỏng client thứ hai.
- Cấu hình timeout OkHttp đã có trong source nhưng chưa có một phép đo riêng, đầy đủ cho server phản hồi chậm hoặc mạng không ổn định theo nhiều mức độ.
- TC09 và benchmark server là các phép đo khác nhau, dữ liệu chưa đủ để kết luận thông lượng ổn định hoặc độ trễ trung bình trong mọi bối cảnh.
- Chiến lược ưu tiên timestamp có thể chịu ảnh hưởng của đồng hồ thiết bị và có thể ghi đè thay đổi hợp lệ của một phía.
- Máy chủ demo chưa có xác thực người dùng, bảo vệ dữ liệu hay triển khai HTTPS công khai; phạm vi phù hợp với bài tập thực nghiệm trong môi trường cục bộ.

7.6 Hướng phát triển

- Thử nghiệm đồng thời trên hai điện thoại thật, thêm stress test và kiểm thử timeout bằng server trì hoãn có kiểm soát.
- Bổ sung thông báo xung đột và khả năng người dùng lựa chọn phiên bản giữ lại.
- Đưa server vào môi trường triển khai có HTTPS và xác thực/ủy quyền khi mở rộng ra nhiều người dùng.
- Thiết kế đồng bộ gia tăng dựa trên mốc phiên bản hoặc thời gian để giảm lượng dữ liệu tải về.
- Đo nhiều lượt trong các mạng khác nhau và bổ sung thống kê độ trễ trung vị, phân vị và tỷ lệ thành công.

CHƯƠNG 8. SẢN PHẨM BÀN GIAO VÀ KẾT LUẬN

8.1 Danh sách sản phẩm

Thành phần	Vị trí / minh chứng
Mã nguồn Android	Android_Source/; GitHub repository.
Máy chủ FastAPI	FastAPI_Server/main.py; FastAPI_Server/README.md; SQLite demo.
APK	APK/app-debug.apk.
Bộ ảnh minh chứng	MinhChung_KiemThu/ (32 ảnh).
Báo cáo Android Test	KetQua_KiemThu_Android/debug/index.html.
Kết quả API Test / Benchmark	KET_QUA_BACKEND.txt; KET_QUA_BENCHMARK_API.csv.
Tài liệu cài đặt	README.md; README_CHAY_VA_TEST_CODE.md.
Video demo	https://drive.google.com/file/d/1ZpKOfXn_und923KkA6hpTPUuKTHu9Mdq/view?usp=drive_link
Báo cáo PDF	524H0117_524H0119.pdf

8.2 Hướng dẫn tái hiện các kịch bản cốt lõi

Để chạy thử, giải nén ZIP; mở FastAPI_Server, tạo môi trường Python và chạy Uvicorn; mở /health và /docs trên trình duyệt; khởi động Android Emulator rồi bật ADB reverse; chạy ứng dụng Android. Sau khi thử thêm công việc online, có thể ngắt đường reverse tcp:8000 để mô phỏng mất kết nối API và tiếp tục tạo/sửa trong Room. Khôi phục ADB reverse và quan sát đồng bộ trong UI, Swagger và log SyncMetrics.

```
& "$env:LOCALAPPDATA\Android\Sdk\platform-tools\adb.exe" reverse --remove tcp:8000
```

```
    & "$env:LOCALAPPDATA\Android\Sdk\platform-tools\adb.exe"
reverse tcp:8000 tcp:8000
    & "$env:LOCALAPPDATA\Android\Sdk\platform-tools\adb.exe"
logcat -d -s SyncMetrics:I
```

Các bước chi tiết và cách chạy Android/Backend tests nằm trong README_CHAY_VA_TEST_CODE.md. Việc ngắt ADB reverse chỉ mô phỏng đường đến API bị mất; để chứng minh hành vi khi Android thật sự mất Internet, cần kiểm tra thêm các bài thử phù hợp.

8.3 Địa chỉ GitHub và video demo

GitHub: https://github.com/nguyengiaphat695-coder/524H0117_524H0119_GiuaKi_DeTai1.git

Video demo: https://drive.google.com/file/d/1ZpKOfXn_und923KkA6hpTPUuKTHu9Mdq/view?usp=drive_link

Lưu ý quyền chia sẻ: nếu repository vẫn để Private, người xem cần được cấp quyền tương ứng. Với video Google Drive, đặt người có đường link có thể xem (Viewer) nếu phù hợp quy định của lớp và bảo đảm không xuất hiện thông tin nhạy cảm.

8.4 Kết luận

Nhóm đã xây dựng ứng dụng quản lý công việc Android theo hướng Offline-First với nguồn dữ liệu giao diện là Room, đồng bộ hai chiều qua Retrofit tới FastAPI, tự động lập lịch bằng WorkManager và các cơ chế kiểm soát xung đột bằng phiên bản, xóa logic và idempotency. Phương pháp triển khai cho phép người dùng tiếp tục thao tác khi tạm mất kết nối với API và đưa dữ liệu về trạng thái nhất quán trong các kịch bản đã thực nghiệm.

Bộ kiểm thử Android và Backend ghi nhận lần lượt 23/23 và 26/26 trường hợp đạt; nhóm cũng lưu 32 ảnh minh chứng cho TC01–TC09. Trong một lượt TC09, 20 bản ghi được đồng bộ trong 2,165 giây. Từ những kết quả này, nhóm rút ra rằng kiến

trúc Offline-First phù hợp cho các ứng dụng cần hoạt động linh hoạt khi đường truyền không ổn định, đồng thời nhận thức rõ các hạn chế của xử lý xung đột theo timestamp, môi trường thử nghiệm cục bộ và phạm vi kiểm thử hiện tại.

TÀI LIỆU THAM KHẢO

[1] Android Developers, “Build an offline-first app”, https://developer.android.com/topic/architecture/data-layer/offline-first

[2] Android Developers, “Guide to app architecture”, https://developer.android.com/topic/architecture

[3] Android Developers, “Save data in a local database using Room”, https://developer.android.com/training/data-storage/room

[4] Android Developers, “WorkManager”, https://developer.android.com/topic/libraries/architecture/workmanager

[5] Kotlin, “Coroutines”, https://kotlinlang.org/docs/coroutines-overview.html

[6] Square, “Retrofit”, https://square.github.io/retrofit/

[7] FastAPI, “FastAPI Documentation”, https://fastapi.tiangolo.com/

[8] SQLite, “SQLite Documentation”, https://www.sqlite.org/docs.html